

INTEGRACIÓN DE INTELIGENCIA ARTIFICIAL EN LIDERABOT

Introducción

La incorporación de la inteligencia artificial en los entornos educativos representa una oportunidad para desarrollar herramientas capaces de acompañar los procesos de aprendizaje de una manera más dinámica y personalizada. En este contexto, LideraBot se plantea como un chatbot educativo desarrollado dentro de un proyecto de diseño de software educativo, utilizando NestJS como framework para el backend y MongoDB como sistema de almacenamiento de información. La integración de un modelo de inteligencia artificial permitiría transformar el chatbot en un asistente capaz de comprender las preguntas de los estudiantes, generar explicaciones adaptadas a sus necesidades y conservar información relevante de sus interacciones.

La propuesta no consiste simplemente en conectar una interfaz de chat con un modelo de inteligencia artificial. Para que LideraBot tenga una función educativa real, la inteligencia artificial debe integrarse con la arquitectura existente del sistema. De esta manera, NestJS actuaría como intermediario entre el usuario, la base de datos y el servicio de inteligencia artificial, mientras MongoDB conservaría las conversaciones, los mensajes, los contenidos educativos y, cuando corresponda, el progreso de los estudiantes. Esta arquitectura permitiría construir un sistema organizado, escalable y apropiado para un proyecto académico de software educativo.

1. Arquitectura general de la solución

La arquitectura propuesta se basa en una comunicación organizada entre cuatro componentes principales: el frontend, el backend desarrollado con NestJS, MongoDB y el servicio externo de inteligencia artificial. El estudiante inicia una conversación desde el frontend y envía una pregunta al backend mediante una petición HTTP. NestJS recibe esta solicitud y la procesa mediante un controlador y un servicio especializado de LideraBot. Antes de solicitar una respuesta a la inteligencia artificial, el backend puede consultar MongoDB para obtener información relevante, como el historial de la conversación, el grado del estudiante, el contenido que está estudiando o su progreso.

Una vez construido el contexto necesario, NestJS envía la información al servicio de inteligencia artificial mediante una API. El modelo genera una respuesta de acuerdo con las instrucciones educativas definidas para LideraBot. Posteriormente, el backend almacena la respuesta en MongoDB y la devuelve al frontend. El flujo general puede resumirse de la siguiente manera: estudiante → frontend → controlador de NestJS → servicio de LideraBot → MongoDB y servicio de IA → almacenamiento de la respuesta → frontend.

Una característica importante de esta arquitectura es que la clave de acceso al servicio de inteligencia artificial no debe encontrarse en el frontend. La comunicación con la API debe realizarse desde el servidor, utilizando variables de entorno. Esto protege las credenciales y permite que el backend controle qué información se envía al modelo.

2. Preparación del proyecto

Antes de integrar la inteligencia artificial, el proyecto debe contar con Node.js, npm, NestJS, MongoDB o MongoDB Atlas y un entorno de desarrollo como Visual Studio Code. También resulta conveniente utilizar Git para controlar las versiones del proyecto. Si el sistema ya dispone de módulos como users, bots, contents, conversations, messages, progress, results y assessment, estos pueden conservarse y aprovecharse durante la integración.

La instalación del SDK de inteligencia artificial se puede realizar desde la raíz del proyecto mediante el comando:

3. Configuración de variables de entorno

Para administrar de forma segura las credenciales se recomienda instalar el módulo de configuración de NestJS. Después de su instalación, el proyecto puede utilizar un archivo .env ubicado en la raíz. En este archivo se pueden definir variables como OPENAI_API_KEY, MONGODB_URI y PORT. El archivo .env debe agregarse al .gitignore para evitar que las credenciales sean publicadas accidentalmente en un repositorio.

La configuración global de NestJS se realiza mediante ConfigModule.forRoot({ isGlobal: true }). Posteriormente, ConfigService permite recuperar las variables de entorno desde los diferentes servicios del proyecto. Esta separación entre configuración y código facilita el mantenimiento y permite cambiar las credenciales o la conexión a MongoDB sin modificar directamente la lógica de la aplicación.

4. Creación del servicio de inteligencia artificial

La integración puede organizarse mediante un nuevo módulo denominado ai. Este módulo puede contener ai.module.ts y ai.service.ts. El servicio AiService tendrá como responsabilidad comunicarse con la API de inteligencia artificial. Para ello, se instala el SDK oficial correspondiente y se crea una instancia utilizando la clave almacenada en ConfigService.

El servicio debe exponer un método, por ejemplo generateResponse(), que reciba la pregunta del estudiante y, opcionalmente, un contexto educativo. El método envía las instrucciones del sistema y la entrada del usuario al modelo mediante la API de Responses. La respuesta textual obtenida se devuelve a LideraBot para continuar con el procesamiento.

Una ventaja de crear un servicio independiente es que la lógica relacionada con la inteligencia artificial queda aislada del resto del sistema. En consecuencia, el controlador no necesita conocer los detalles de la API externa y el servicio de LideraBot puede concentrarse en coordinar usuarios, conversaciones, contenidos, progreso y mensajes.

5. Integración con MongoDB y Mongoose

MongoDB desempeña un papel fundamental porque permite conservar la información necesaria para construir una experiencia conversacional. Si el proyecto ya utiliza @nestjs/mongoose, se pueden crear o adaptar esquemas para Conversation y Message. Un mensaje puede contener, como mínimo, el identificador de la conversación, el rol que generó el mensaje —usuario o asistente—, el contenido y las marcas de tiempo.

El esquema de Message puede representar una estructura conceptual como la siguiente: conversationId identifica la conversación; role indica si el mensaje pertenece al estudiante o a

LideraBot; content contiene el texto; y createdAt permite ordenar cronológicamente la conversación. El MessagesService puede ofrecer métodos para crear mensajes y recuperar el historial asociado a una conversación.

La separación entre conversaciones y mensajes permite que una conversación tenga múltiples interacciones sin almacenar toda la información en un único documento. Además, facilita las consultas y permite recuperar solamente los mensajes necesarios para generar el contexto de una nueva respuesta.

6. Creación del servicio de LideraBot

El servicio principal de LideraBot constituye el punto de coordinación entre los diferentes componentes. Su método chat() puede recibir un conversationId y el mensaje escrito por el estudiante. Primero se almacena la pregunta en MongoDB. Luego se recuperan los mensajes anteriores de la conversación y se construye un contexto organizado.

Con este contexto, LideraBot solicita una respuesta al AiService. Cuando la inteligencia artificial devuelve el resultado, el servicio almacena nuevamente la respuesta en MongoDB con el rol assistant. Finalmente, devuelve al frontend un objeto que contiene la respuesta generada.

Este proceso permite que cada interacción quede registrada. Por ejemplo, ante la pregunta «¿Qué es una variable?», el estudiante puede recibir una explicación inicial y, en una segunda interacción, preguntar por un ejemplo. El historial almacenado permite que LideraBot conozca el contenido de la conversación anterior y mantenga una respuesta coherente.

7. Creación del endpoint para el chatbot

Para que el frontend pueda comunicarse con LideraBot, NestJS debe exponer un endpoint HTTP. Una opción sencilla consiste en utilizar POST /bots/chat. El cuerpo de la petición puede incluir conversationId y message. El BotsController recibe estos datos y los entrega a LideraBotService.

También resulta recomendable utilizar DTOs y validaciones mediante class-validator. De esta manera, el backend puede comprobar que conversationId y message sean cadenas de texto y que no estén vacíos. Esta validación evita que solicitudes incompletas o malformadas lleguen hasta el servicio de inteligencia artificial.

8. Diseño del comportamiento educativo

La principal diferencia entre un chatbot convencional y LideraBot debe encontrarse en su propósito pedagógico. La inteligencia artificial necesita recibir instrucciones claras que definan su función como asistente educativo. En lugar de limitarse a proporcionar respuestas, LideraBot puede explicar conceptos de manera sencilla, utilizar ejemplos, adaptar el lenguaje al nivel del estudiante, dividir los problemas complejos en pasos y formular preguntas para comprobar la comprensión.

Las instrucciones también pueden establecer que el sistema no debe inventar información, debe mantener un lenguaje respetuoso y debe evitar entregar directamente respuestas de evaluaciones cuando el propósito sea promover el razonamiento. De esta forma, la IA se convierte en un recurso de acompañamiento y no simplemente en un mecanismo para resolver actividades.

9. Personalización mediante usuarios, contenidos y progreso

Una de las posibilidades más importantes de la integración consiste en utilizar los datos que ya maneja el software educativo. Si MongoDB contiene información del estudiante, los contenidos disponibles y su progreso, LideraBot puede construir un contexto personalizado antes de consultar al modelo.

Por ejemplo, si un estudiante de séptimo grado se encuentra trabajando en programación básica y presenta dificultades con las variables, el backend puede obtener el grado, el tema actual, el contenido relacionado y el progreso registrado. Estos datos se transforman en información relevante para el modelo. La inteligencia artificial podrá entonces adaptar la explicación al nivel correspondiente.

Este mecanismo debe utilizar únicamente la información necesaria. No resulta conveniente enviar toda la base de datos al modelo. El backend debe filtrar los datos, seleccionar el contexto pertinente y enviar solamente aquello que contribuya a responder la pregunta. Esta práctica mejora la organización, reduce el volumen de información procesada y protege los datos que no son necesarios para la interacción.

10. Memoria de las conversaciones

La memoria conversacional puede implementarse inicialmente mediante MongoDB, aprovechando los módulos `conversations` y `messages`. Cada conversación puede estar relacionada con un usuario y contener múltiples mensajes. Para evitar enviar historiales excesivamente extensos a la inteligencia artificial, el backend puede recuperar solamente los últimos mensajes relevantes.

Una estrategia inicial consiste en utilizar los últimos diez mensajes de una conversación. Más adelante, el proyecto podría incorporar mecanismos de resumen para conservar los aspectos importantes de conversaciones largas sin enviar todo el historial al modelo. De esta manera, LideraBot podría mantener continuidad sin aumentar innecesariamente la cantidad de información procesada.

11. Manejo de errores y seguridad

La integración debe contemplar situaciones en las que la API de inteligencia artificial no esté disponible, se produzca un error de conexión o la solicitud no pueda ser procesada. El `AIService` debe manejar estas excepciones y devolver una respuesta controlada. El backend también puede registrar los errores para facilitar su diagnóstico.

La seguridad constituye otro aspecto fundamental. Las claves de API no deben escribirse directamente en los archivos `TypeScript` ni enviarse al frontend. Deben mantenerse en variables de entorno y protegerse mediante el archivo `.gitignore`. Asimismo, los datos de los estudiantes deben tratarse de acuerdo con las necesidades del proyecto y únicamente debe enviarse al modelo la información estrictamente necesaria para generar la respuesta educativa.

12. Pruebas de funcionamiento

Una vez implementados los componentes, la integración puede probarse inicialmente mediante Postman o Swagger. El servidor se ejecuta con `npm run start:dev` y se realiza una petición

POST al endpoint /bots/chat. El cuerpo puede contener un identificador de conversación y una pregunta como «¿Qué es una variable?».

El resultado esperado es una respuesta JSON con el texto generado por LideraBot. Al mismo tiempo, MongoDB debe registrar tanto la pregunta del estudiante como la respuesta del asistente. Una segunda petición dentro de la misma conversación permitirá comprobar si el historial está siendo recuperado correctamente. Estas pruebas deben realizarse antes de conectar el frontend para identificar problemas en el backend de forma aislada.

13. Integración final con el frontend

Después de comprobar que el backend responde correctamente, el frontend puede conectarse al endpoint del chatbot. La interfaz debe enviar el mensaje del estudiante mediante una petición HTTP y mostrar la respuesta recibida. También puede conservar el conversationId para que varias preguntas pertenezcan a la misma conversación.

El flujo final será: el estudiante escribe un mensaje; el frontend realiza una petición a NestJS; BotsController recibe la solicitud; LideraBotService guarda y recupera información; AiService consulta al modelo; la respuesta se almacena en MongoDB y finalmente se devuelve al frontend. Esta separación permite que la interfaz se mantenga independiente de los detalles internos de la inteligencia artificial.

Conclusión

La integración de inteligencia artificial en LideraBot puede convertir el chatbot desarrollado para el proyecto de software educativo en una herramienta de acompañamiento más completa. NestJS permite organizar la comunicación entre el frontend, MongoDB y el servicio de inteligencia artificial mediante controladores, servicios y módulos independientes. MongoDB, por su parte, permite conservar las conversaciones, los mensajes, los contenidos y la información de progreso que puede utilizarse para personalizar la experiencia.

El aspecto más relevante de la propuesta es que la inteligencia artificial no se considera un componente aislado. Su valor aparece cuando se integra con la información educativa administrada por el sistema. Al utilizar el historial de conversaciones, los contenidos y el progreso de forma controlada, LideraBot puede ofrecer explicaciones adecuadas al contexto del estudiante y favorecer procesos de aprendizaje basados en la comprensión y el razonamiento.

La implementación puede realizarse progresivamente: primero se verifica la comunicación básica con la API de inteligencia artificial; después se incorpora el almacenamiento en MongoDB; posteriormente se integra el contexto educativo y, finalmente, se conecta el frontend. Esta metodología reduce la complejidad y permite comprobar cada componente antes de avanzar al siguiente. Como resultado, el proyecto obtiene una arquitectura clara, mantenible y adecuada para demostrar cómo la inteligencia artificial puede integrarse de manera responsable dentro de un software educativo.

Referencias

OpenAI. (s. f.). Developer documentation: API y Responses. <https://platform.openai.com/docs/>

NestJS. (s. f.). Documentation: Configuration. <https://docs.nestjs.com/techniques/configuration>

NestJS. (s. f.). Documentation: MongoDB. <https://docs.nestjs.com/techniques/mongodb>

NestJS. (s. f.). Documentation: Controllers. <https://docs.nestjs.com/controllers>

Apéndice A. Comandos y fragmentos esenciales de implementación

Instalación de dependencias

```
npm install openai
npm install @nestjs/config
npm install @nestjs/mongoose mongoose
```

Variables de entorno

```
OPENAI_API_KEY=TU_API_KEY
MONGODB_URI=TU_URI_DE_MONGODB
PORT=3000
```

Creación de módulos y servicios

```
nest g module ai
nest g service ai
```

Endpoint de prueba

```
POST http://localhost:3000/bots/chat

{
  "conversationId": "123",
  "message": "¿Qué es una variable?"
}
```